Do Students Agree with LLM Feedback on Their Own Code? A Within-Subject Study of Four Feedback Configurations in Software Project Courses

Phuc Sanh Nguyen, Song Huong Dang, Quang Hieu Le, Viet Bao Nguyen, and Phuc Thinh Do(✉)

FPT University, Ho Chi Minh City, Vietnam
npsdn1906@gmail.com, dangsonghuong010405@gmail.com, lequanghieu2504@gmail.com, babluokei4@gmail.com, ThinhDP2@fe.edu.vn

Abstract. Oral defense (viva) in software engineering capstone projects is essential for verifying students' comprehension of business logic and codebase architecture, yet manual grading is labor-intensive and inconsistent, whereas off-the-shelf large language models (LLMs) risk code-context hallucinations, prompt injection, and sycophantic grading. This paper presents the pedagogical evaluation and feedback engine of **PACE**, an AI-assisted viva assessment system combining complexity-aware hybrid ground-truth extraction (routing raw source code versus Mermaid control/data-flow graphs at a function-node threshold of $\tau = 10$) with a three-layer safety guardrail architecture and XML-based delimiter sandboxing. To mitigate single-model grading sycophancy, we propose a role-decoupled **Multi-LLM Ensemble** (*Researching* mode) sequentially chaining **DeepSeek-V3** (*Code Inspector*) with **Groq (`openai/gpt-oss-120b`)** (*Pedagogical Synthesizer and Rubric Grader*), evaluated alongside three single-LLM formative feedback formats (*Sandwich*, *Rubric*, and *Explainable*). In a randomized, student-blinded, within-subject experiment at FPT University, 86 students across 28 capstone repositories submitted 344 scenario-based viva responses ($k = 4$ per student) and rated each feedback instance using a 15-item, five-point Likert-scale instrument with unidirectional positive keying, adapted from the Feedback Perceptions Questionnaire (FPQ), explainable AI (XAI), argument persuasiveness, psychological safety, and TAM/UTAUT frameworks. Across 344 ratings, the proportions of positive student responses (ratings of 4 or 5 out of 5) endorsed AI judgment (`ACCEPTANCE`: 79.9%), future use intention (`BEHAVIORAL INTENTION`: 68.0%), and learning-related encouragement and self-reported motivation (`AFFECT`: 41.9%), yielding an overall multidomain composite mean of 4.07/5 (95% CI [3.90, 4.23]). All six repository-level 95% bootstrap confidence intervals for direct acceptance (`ACCEPTANCE`) included zero without establishing statistical equivalence, and observational raw-versus-graph acceptance remained inconclusive ($b = 0.03$, 95% CI $[-0.18, 0.26]$). Exploratory analysis revealed a 0.37-point actionability (`ACTIONABILITY`) difference between *Sandwich* and *Explainable* (approximate simultaneous 95% sensitivity interval [0.02, 0.72] across 90 paired item-level contrasts derived from 6 format pairs × 15 criteria) without isolating the independent effect of the `next_steps` component. These findings reflect learners' self-reported perceptions rather than expert-verified accuracy, actual revision behavior, or learning gains.

Keywords: Automated Viva Assessment · Multi-LLM Ensemble · Formative Feedback · Pedagogical AI Guardrails · Software Engineering Education.

1 Introduction

Formative feedback is widely recognized as an important factor in improving student learning. However, its effectiveness depends largely on how learners understand, interpret, and act on the guidance they receive [11,30]. Technically accurate feedback does not necessarily lead to better understanding or meaningful revisions [1,33]. Research on feedback literacy emphasizes that students need to interpret feedback, make judgments about their own work, and use the feedback to guide their actions. These activities are essential parts of the feedback process, rather than simply outcomes that occur after feedback is provided [1,33]. In human-computer interaction, users' acceptance of automated feedback and willingness to act on it are also influenced by its perceived usefulness [4,32], as well as by psychological factors such as trust and users' tendency to accept or reject algorithmic judgments [22,6,26].

With the rapid development of generative artificial intelligence, large language models (LLMs) are increasingly used in computing education to identify code issues and provide formative feedback at scale [16,5,3]. However, general-purpose LLMs may generate inaccurate feedback when they lack sufficient context about the actual student codebase, including cases in which they misinterpret code or produce unsupported explanations [13,19]. In addition, deploying LLMs in educational assessment requires appropriate safeguards to maintain pedagogical boundaries and reduce the risk of misuse, including prompt injection attacks [25]. Recent studies in programming education and higher education have also found that students' perceptions of AI-generated feedback vary across several dimensions, including technical accuracy, level of detail, the availability of supporting evidence, and trust in comparison with feedback from human experts [9,34,27,8].

Two research gaps motivate this study:

- **Gap 1 (Automated Viva Assessment and Hybrid Ground-Truth Contexts in Team Capstone Projects):** Software engineering capstone projects often involve shared codebases developed collaboratively by student teams. Traditional input-output autograders can assess whether a program produces the expected results, but they are less suited to determining whether individual students understand the code and its architecture. Although asking students questions about their own programs has been explored as a way to assess code understanding [24], the use of automated, scenario-based oral defenses (viva) that combine multi-language static analysis with complexity-aware routing between different code representations remains underexplored.
- **Gap 2 (Formative Feedback Formats and Single-Model Sycophancy):** Using a single LLM as an automated evaluator (*LLM-as-a-Judge*) can introduce biases, including a tendency to favor certain responses or follow patterns that affect grading consistency [35]. In practical systems, AI-generated feedback may combine praise, rubric-based assessment, code evidence, and model orchestration in different configurations. These configurations reflect different approaches to formative feedback:

the *Sandwich* format balances positive and corrective comments [28]; *Rubric* uses explicit scoring criteria to support consistent evaluation [15]; *Explainable* emphasizes evidence and transparency [18]; and the *Multi-LLM Ensemble* uses multiple models in coordinated roles to improve evaluation quality and reduce dependence on a single model's judgment [7,35]. However, comparative within-subject studies of these feedback formats, based on students' actual capstone project codebases and focused on learners' perceptions, remain limited.

To address these gaps, we present **PACE** (*Project Assistance and Course Evaluation*), an AI-assisted viva assessment system deployed at FPT University. PACE uses multi-language static analysis tools, including Roslyn for C#/.NET and Tree-sitter for Java, JavaScript, and TypeScript, to extract control-flow and data-flow information and represent it as graphs. A complexity-aware context router selects raw source code (`Raw`) when the number of distinct function nodes, N_w, is at most 10 ($N_w \leq 10$), and Mermaid flow graphs (`Graph`) when $N_w > 10$. These representations are used to generate scenario-based questions grounded in the project code through OpenAI `gpt-6-luna`. To support safe evaluation and feedback generation, PACE incorporates a three-layer safety guardrail architecture comprising input, execution, and output guardrails, together with XML-based delimiter sandboxing. The delimiters distinguish the ground-truth context, exam question, suggested answer, and student answer using the tags <ground_truth_context>, <exam_question>, <suggested_answer>, and <student_answer>. These safeguards are designed to reduce the risk of prompt injection and keep the evaluation process aligned with the intended task. PACE supports four formative feedback formats: three single-LLM configurations powered by DeepSeek-V3 (*Sandwich*, *Rubric*, and *Explainable*) and a role-decoupled *Researching* mode, referred to as the *Multi-LLM Ensemble*. In this mode, DeepSeek-V3 acts as the *Code Inspector*, while Groq (`openai/gpt-oss-120b`) serves as the *Pedagogical Synthesizer and Rubric Grader*.

We conducted a randomized, student-blinded, within-subject experiment in which students answered scenario-based questions about business rules in different modules of their team's project repositories. The feedback formats were randomly assigned without replacement, and students rated each feedback instance using a 15-item, five-point Likert-scale instrument in which all items were scored in the same direction. The instrument was developed with reference to the Feedback Perceptions Questionnaire (FPQ), research on explainable AI (XAI) and argument persuasiveness, psychological safety, and the Technology Acceptance Model (TAM) and Unified Theory of Acceptance and Use of Technology (UTAUT). We also considered established methods for assessing measurement reliability and identifying potential response and method biases [2,12,29,20,31]. This study addresses three research questions:

RQ1 How do students evaluate the content quality, perceived usefulness, and overall acceptance of LLM-generated feedback on viva responses about their own capstone project codebases?

RQ2 How do students' agreement rates and multidomain ratings differ across the four feedback formats (*Sandwich*, *Rubric*, *Explainable*, and *Researching/Multi-LLM Ensemble*) based on non-parametric tests and pairwise comparisons with corrections for multiple testing [10,14,21]?

RQ3 To what extent are student acceptance and perceived feedback quality associated with the use of raw source-code contexts ($N_w \leq 10$) versus Mermaid flow-graph contexts ($N_w > 10$) in the deployed system?

The main contribution of this work is an integrated AI-assisted viva assessment system with multiple safety safeguards, together with an empirical evaluation of different formative feedback formats for capstone project defense preparation. This study extends existing approaches to automated program questioning [24] to team-based project defense and examines students' agreement with AI judgments separately from their perceptions of feedback actionability, learning-related motivation, and intention to use the system. The findings represent learners' self-reported perceptions under a controlled experimental design and statistical analysis. They do not directly establish expert-verified technical accuracy, actual revision behavior, or long-term improvements in learning outcomes.

The remainder of this paper is structured as follows: Section 2 reviews related literature; Section 3 presents the methodology and architecture; Section 4 details the experimental evaluation; Section 5 discusses findings; Section 6 examines threats to validity; and Section 7 concludes the paper.

2 Related Work

This work draws on three main research areas: automated formative feedback in computing education, student acceptance and perceptions of AI-generated guidance, and pedagogical feedback configurations in software engineering assessment.

2.1 LLM-Generated Feedback in Programming Education

Automated assessment in computing education predates the emergence of generative AI, with early diagnostic tools focusing primarily on static error detection rather than actionable next steps [17]. While recent evaluations demonstrate that off-the-shelf LLMs can effectively identify issues in code, they remain susceptible to hallucinating nonexistent defects or suggesting misleading fixes [13,19,3]. To address these safety and reliability concerns, specialized systems such as *CodeHelp* have demonstrated the potential of incorporating guardrails to provide scalable programming assistance without directly supplying complete solutions [25]. Building on these foundations, our approach shifts the evaluation focus from isolated, beginner-level coding snippets to comprehensive explanations of business logic and system architecture across evolving, multi-language team repositories.

2.2 Student Perceptions and Acceptance of AI Feedback

Understanding how learners respond to automated feedback requires examining established technology acceptance and trust frameworks. According to the Technology Acceptance Model (TAM) and UTAUT, perceived usefulness and ease of use are important predictors of technology acceptance and use [4,32], whereas appropriate reliance on automation requires calibrating trust to actual system reliability [22]. Psychological

phenomena such as algorithm aversion, in which users prematurely abandon algorithmic recommendations after observing errors, coexist with instances of algorithm appreciation [6,26]. In computing education, empirical studies have revealed diverse learner preferences and perceptions: Zhang et al. documented student preferences for feedback that explicitly references source code [34], Er et al. identified continued advantages of human instructor feedback [8], and Nazaretsky et al. demonstrated that student trust is sensitive to the perceived source of AI-generated feedback [27]. These findings highlight the importance of evaluating multiple dimensions of learner perception, including acceptance, behavioral intention, trust, and affective responses, alongside technical correctness.

2.3 Pedagogical Feedback Configurations and Multi-Model Orchestration

Different formats of formative feedback are grounded in distinct pedagogical principles. Sandwich feedback (*Sandwich*), which places constructive criticism between supportive remarks, has been shown in controlled studies to improve learners' affective responses, even when performance gains remain modest [28]. Explicit scoring rubrics (*Rubric*) establish transparent evaluation criteria and promote grading consistency [15], while evidence-based explanations (*Explainable*) can support appropriate trust calibration by providing transparent references to source code [18]. To mitigate grading biases and sycophancy observed in LLM-based evaluations (*LLM-as-a-Judge*) [35], multi-agent debate and multi-model coordination architectures have been proposed to improve factual accuracy and objectivity in reasoning [7,35]. However, existing literature offers limited evidence from rigorous, randomized within-subject evaluations comparing these distinct formative feedback approaches, particularly when implemented through role-decoupled multi-model ensembles operating on authentic student capstone codebases.

Position Relative to Prior Literature. While prior work has used automated comprehension checks to assess program understanding [24], our work extends this approach to a structured viva defense preparation workflow involving collaborative team repositories. Unlike studies focusing on isolated CS1 programming tasks or hypothetical model-generated code snippets [34,23], PACE investigates student perceptions of multiple AI feedback formats in the context of authentic, multi-language capstone projects. Furthermore, by implementing a complexity-aware hybrid context router ($\tau = 10$), a three-layer guardrail architecture with XML delimiter sandboxing, and a role-decoupled Multi-LLM Ensemble, this study provides an empirical evaluation of guardrailed, multi-format formative feedback in software engineering education.

3 The PACE System

Figure 1 summarizes the PACE pipeline, including repository analysis, complexity-aware context routing, question generation, and multi-format pedagogical evaluation.

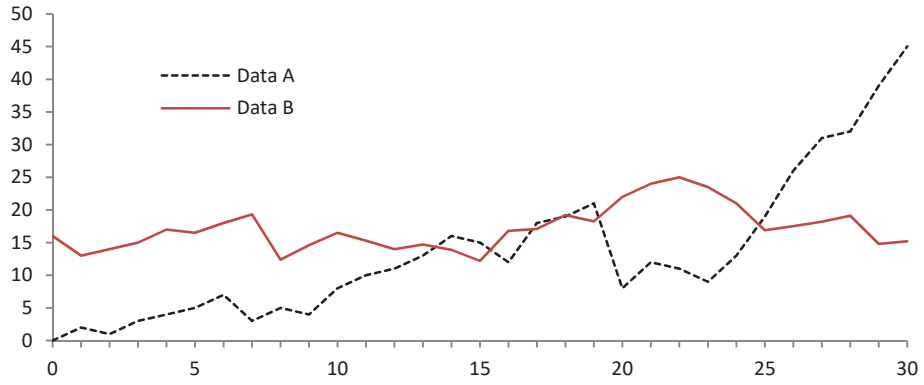


Fig. 1. Overview of the PACE system pipeline, from repository analysis to multi-format feedback.

3.1 Multi-Language Static Analysis and Complexity-Aware Context Routing

For each submitted team repository, PACE clones the target branch and parses C#/.NET source code using Roslyn and Java (Spring Boot), JavaScript, and TypeScript (Express, NestJS, Hono) source code using tree-sitter. Extracted API endpoints and controllers are grouped into user-facing *business functions* (e.g., "budget management" or "asset auction") through metadata analysis performed by Gemini 3.1 Flash Lite, subject to instructor review and confirmation.

To assess code complexity and select an appropriate context representation, PACE computes the number of distinct function nodes N_w in each business function's call graph, derived from entry points and directed call relationships. A complexity-aware hybrid router then selects the context representation. When $N_w \leq 10$, the system retrieves raw source code (`Raw`) to preserve fine-grained details of conditional expressions. When $N_w > 10$, it uses a Mermaid control- and data-flow graph (`Graph`) containing ordered edges, branch conditions, and return types to reduce token overhead and syntactic noise. Scenario-based questions are generated for each business function using OpenAI `gpt-6-luna` (temperature 0.3, strict JSON schema output). Question generation follows a business-language-only constraint that prohibits technical keywords (such as *Controller*, *DTO*, or *SQL*). Questions are organized into three difficulty levels based on exception branches: Happy Path, Single Exception, and Double Exception.

3.2 Guardrailed Multi-Format Evaluation Engine and Multi-LLM Ensemble

When a student submits $k = 4$ scenario answers across different business functions, PACE uses an atomic database transaction (< 200 ms) with row-level locking (`SELECT FOR UPDATE`) to maintain data integrity before invoking external LLMs. Each submission is processed through a three-layer AI guardrail architecture: (1) an *Input Guardrail* that applies 17 blacklist-based checks for prompt injection and jailbreak attempts in English and Vietnamese; (2) an *Execution Guardrail* that blocks malicious scripts and binary

payloads using compiled regular expressions and sanitizes Unicode control characters; and (3) an *Output Guardrail* that validates strict JSON structure and checks for potential API key leakage.

To reduce the risk of indirect prompt injection and reinforce the instruction hierarchy, the evaluation engine separates input components using XML delimiters:

```
<ground_truth_context> {Raw code or Mermaid graph} </ground_truth_context>
<exam_question> {Sanitized scenario question} </exam_question>
<suggested_answer> {Reference suggested answer} </suggested_answer>
<student_answer> {Sanitized student answer} </student_answer>
```

The system supports four feedback formats, assigned through a random permutation without replacement within each student ($\mathcal{F}$ = {Sandwich, Rubric, Explainable, Researching}), ensuring that every student encounters each configuration exactly once:

- **Sandwich**: Feedback structured into positive reinforcement, constructive criticism, and actionable next steps, generated by DeepSeek-V3 (`deepseek-chat`, temperature 0.2).
- **Rubric**: Quantitative feedback assessing technical competencies using a five-point scoring scale and detailed commentary, generated by DeepSeek-V3.
- **Explainable**: Evidence-based feedback incorporating explicit source-code citations (`code_evidence`) and explanations of logical reasoning (`logic_explanation`), generated by DeepSeek-V3.
- **Researching (Multi-LLM Ensemble)**: A role-decoupled, two-stage architecture designed to mitigate grading biases and sycophancy associated with single-model evaluation. In Stage 1, DeepSeek-V3 serves as a *Code Inspector*, extracting code evidence and identifying logical inconsistencies. In Stage 2, Groq (`openai/gpt-oss-120b`) serves as a *Pedagogical Synthesizer and Rubric Grader*, integrating the inspection report from DeepSeek-V3 to generate comprehensive pedagogical feedback and rubric scores. A multistage fault-tolerance mechanism handles rate-limit errors (HTTP 429) through exponential backoff and falls back to single-model processing if downstream validation fails.

3.3 Empirical Data Collection and Survey Instrumentation

After receiving feedback, students evaluate each feedback instance through a web interface using a 15-item questionnaire with a five-point Likert response scale (1 = Strongly disagree to 5 = Strongly agree). All items are positively keyed. Drawing on the Feedback Perceptions Questionnaire (FPQ), explainable AI trustworthiness, argument persuasiveness, psychological safety, and TAM/UTAUT frameworks, the 15 items assess information and pedagogical quality (`ACCURACY`, `SPECIFICITY`, `CLARITY`), explainability (`TRANSPARENCY`), usefulness and actionability (`USEFULNESS`, `ACTIONABILITY`), trust and fairness (`COMPETENCE TRUST`, `RELIABILITY`, `FAIRNESS`), affect and psychological safety (`AFFECT`, `PSYCHOLOGICAL SAFETY`), and behavioral acceptance (`ACCEPTANCE`, `PERSUASIVENESS`, `WILLINGNESS`, `BEHAVIORAL INTENTION`). The data export pipeline automatically excludes load-testing accounts (identified by MSSV values beginning with `K6`) to ensure that the resulting datasets contain only eligible student responses.

4 Empirical Evaluation and Survey Design

Describe the experimental setting, participant demographics, questionnaire instrumentation, and procedure.

4.1 Instrumentation and Construct Measurements

Table 1. Summary of survey constructs, dimensions, and operational definitions.

Construct	Metric/Code	Operational Definition
Accuracy	ACC	Correctness of technical explanations and error diagnoses.
Clarity	CLA	Ease of comprehension, structure, and readability.
Specificity	SPE	Concrete pointers to lines of code or specific logic.
Transparency	TRA	Explainability of why and how the feedback was derived.
Actionability	ACT	Practical guidance on how to fix or refine the implementation.
Acceptance	ACP	Student agreement and willingness to adopt recommendations.

Table 1 highlights the evaluation dimensions. The numbering of tables and figures continues sequentially throughout the manuscript.

5 Results and Discussion

Present statistical analyses, survey findings, correlation matrices, and thematic student responses.

5.1 Analysis of Survey Constructs (RQ1)

Present findings addressing RQ1. Reference additional figures where necessary:
Fig. 2 illustrates the Likert-scale distributions observed in the participant cohort.

5.2 Cross-Feedback Comparative Analysis (RQ2)

Examine differences among feedback styles or presentation formats.

5.3 Source Modality Comparison (RQ3)

Compare question generation modalities (e.g., codebase extraction vs. knowledge graph).

Fig. 3 depicts the inter-item correlation coefficients.

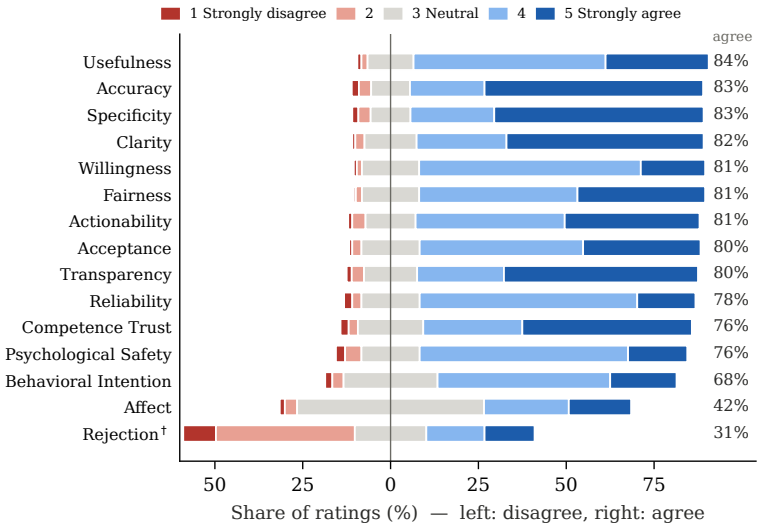


Fig. 2. Distribution of student responses across the Likert-scale items.

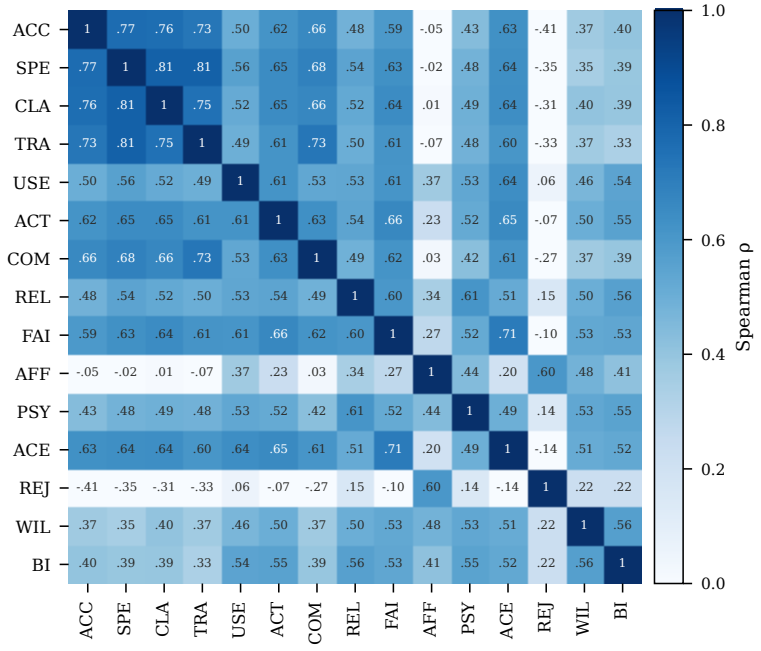


Fig. 3. Correlation matrix across evaluation dimensions and survey questions.

6 Threats to Validity

Discuss construct validity, internal validity, external validity, and reliability considerations.

7 Conclusion and Future Work

Summarize the main takeaways, lessons learned, and future research directions.

Acknowledgments. This research was supported by Project Grant Number XYZ. The authors express their gratitude to the student participants and course instructors for their support during data collection.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

A Survey Instrument Details

Detailed survey questions and Likert-scale coding schemes can be provided here. Continues the same formatting standards as the main text.

References

1. Carless, D., Boud, D.: The development of student feedback literacy: Enabling uptake of feedback. Assessment & Evaluation in Higher Education **43**(8), 1315–1325 (2018). https://doi.org/10.1080/02602938.2018.1463354
2. Cronbach, L.J.: Coefficient alpha and the internal structure of tests. Psychometrika **16**(3), 297–334 (1951). https://doi.org/10.1007/BF02310555
3. Dai, W., Lin, J., Jin, H., Li, T., Tsai, Y.S., Gašević, D., Chen, G.: Can large language models provide feedback to students? A case study on ChatGPT. In: 2023 IEEE International Conference on Advanced Learning Technologies (ICALT). pp. 323–325. IEEE (2023). https://doi.org/10.1109/ICALT58122.2023.00100
4. Davis, F.D.: Perceived usefulness, perceived ease of use, and user acceptance of information technology. MIS Quarterly **13**(3), 319–340 (1989). https://doi.org/10.2307/249008
5. Denny, P., Prather, J., Becker, B.A., Finnie-Ansley, J., Hellas, A., Leinonen, J., Luxton-Reilly, A., Reeves, B.N., Santos, E.A., Sarsa, S.: Computing education in the era of generative AI. Communications of the ACM **67**(2), 56–67 (2024). https://doi.org/10.1145/3624720
6. Dietvorst, B.J., Simmons, J.P., Massey, C.: Algorithm aversion: People erroneously avoid algorithms after seeing them err. Journal of Experimental Psychology: General **144**(1), 114–126 (2015). https://doi.org/10.1037/xge0000033
7. Du, Y., Li, S., Torralba, A., Tenenbaum, J.B., Mordatch, I.: Improving factuality and reasoning in language models through multiagent debate. In: Proceedings of the 41st International Conference on Machine Learning (ICML). Proceedings of Machine Learning Research, vol. 235, pp. 11733–11763. PMLR (2024), https://proceedings.mlr.press/v235/du24e.html
8. Er, E., Akçapınar, G., Bayazıt, A., Noroozi, O., Banihashem, S.K.: Assessing student perceptions and use of instructor versus AI-generated feedback. British Journal of Educational Technology **56**(3), 1074–1091 (2025). https://doi.org/10.1111/bjet.13558

9. Escalante, J., Pack, A., Barrett, A.: AI-generated feedback on writing: Insights into efficacy and ENL student preference. International Journal of Educational Technology in Higher Education **20**, 57 (2023). https://doi.org/10.1186/s41239-023-00425-2
10. Friedman, M.: The use of ranks to avoid the assumption of normality implicit in the analysis of variance. Journal of the American Statistical Association **32**(200), 675–701 (1937). https://doi.org/10.1080/01621459.1937.10503522
11. Hattie, J., Timperley, H.: The power of feedback. Review of Educational Research **77**(1), 81–112 (2007). https://doi.org/10.3102/003465430298487
12. Hayes, A.F., Coutts, J.J.: Use omega rather than Cronbach's alpha for estimating reliability. But. . . . Communication Methods and Measures **14**(1), 1–24 (2020). https://doi.org/10.1080/19312458.2020.1718629
13. Hellas, A., Leinonen, J., Sarsa, S., Koutcheme, C., Kujanpää, L., Sorva, J.: Exploring the responses of large language models to beginner programmers' help requests. In: Proceedings of the 2023 ACM Conference on International Computing Education Research (ICER 2023), Volume 1. pp. 93–105. ACM (2023). https://doi.org/10.1145/3568813.3600139
14. Holm, S.: A simple sequentially rejective multiple test procedure. Scandinavian Journal of Statistics **6**(2), 65–70 (1979)
15. Jonsson, A., Svingby, G.: The use of scoring rubrics: Reliability, validity and educational consequences. Educational Research Review **2**(2), 130–144 (2007). https://doi.org/10.1016/j.edurev.2007.05.002
16. Kasneci, E., Sessler, K., Küchemann, S., Bannert, M., Dementieva, D., Fischer, F., Gasser, U., Groh, G., Günnemann, S., Hüllermeier, E., et al.: ChatGPT for good? On opportunities and challenges of large language models for education. Learning and Individual Differences **103**, 102274 (2023). https://doi.org/10.1016/j.lindif.2023.102274
17. Keuning, H., Jeuring, J., Heeren, B.: A systematic literature review of automated feedback generation for programming exercises. ACM Transactions on Computing Education **19**(1), 3:1–3:43 (2019). https://doi.org/10.1145/3231711
18. Khosravi, H., Buckingham Shum, S., Chen, G., Conati, C., Tsai, Y.S., Kay, J., Knight, S., Martinez-Maldonado, R., Sadiq, S., Gašević, D.: Explainable artificial intelligence in education. Computers and Education: Artificial Intelligence **3**, 100074 (2022). https://doi.org/10.1016/j.caeai.2022.100074
19. Kiesler, N., Lohr, D., Keuning, H.: Exploring the potential of large language models to generate formative programming feedback. In: 2023 IEEE Frontiers in Education Conference (FIE). pp. 1–5. IEEE (2023). https://doi.org/10.1109/FIE58773.2023.10343457
20. Kim, Y., Dykema, J., Stevenson, J., Black, P., Moberg, D.P.: Straightlining: Overview of measurement, comparison of indicators, and effects in mail–web mixed-mode surveys. Social Science Computer Review **37**(2), 214–233 (2019). https://doi.org/10.1177/0894439317752406
21. Lakens, D.: Equivalence tests: A practical primer for *t* tests, correlations, and meta-analyses. Social Psychological and Personality Science **8**(4), 355–362 (2017). https://doi.org/10.1177/1948550617697177
22. Lee, J.D., See, K.A.: Trust in automation: Designing for appropriate reliance. Human Factors **46**(1), 50–80 (2004). https://doi.org/10.1518/hfes.46.1.50_30392
23. Lehtinen, T., Koutcheme, C., Hellas, A.: Let's ask AI about their programs: Exploring ChatGPT's answers to program comprehension questions. arXiv:2404.11734 (2024). https://doi.org/10.48550/arXiv.2404.11734
24. Lehtinen, T., Santos, A.L., Sorva, J.: Let's ask students about their programs, automatically. In: 2021 IEEE/ACM 29th International Conference on Program Comprehension (ICPC). pp. 467–475. IEEE (2021). https://doi.org/10.1109/ICPC52881.2021.00054
25. Liffiton, M., Sheese, B.E., Savelka, J., Denny, P.: CodeHelp: Using large language models with guardrails for scalable support in programming classes. In: Proceedings of the 23rd Koli

Calling International Conference on Computing Education Research. pp. 1–11. ACM (2023). https://doi.org/10.1145/3631802.3631830
26. Logg, J.M., Minson, J.A., Moore, D.A.: Algorithm appreciation: People prefer algorithmic to human judgment. Organizational Behavior and Human Decision Processes **151**, 90–103 (2019). https://doi.org/10.1016/j.obhdp.2018.12.005
27. Nazaretsky, T., Mejia-Domenzain, P., Swamy, V., Frej, J., Käser, T.: AI or human? Evaluating student feedback perceptions in higher education. In: Technology Enhanced Learning for Inclusive and Equitable Quality Education (EC-TEL 2024). Lecture Notes in Computer Science, vol. 15159, pp. 284–298. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-72315-5_20
28. Parkes, J., Abercrombie, S., McCarty, T.: Feedback sandwiches affect perceptions but not performance. Advances in Health Sciences Education **18**(3), 397–407 (2013). https://doi.org/10.1007/s10459-012-9377-9
29. Podsakoff, P.M., MacKenzie, S.B., Lee, J.Y., Podsakoff, N.P.: Common method biases in behavioral research: A critical review of the literature and recommended remedies. Journal of Applied Psychology **88**(5), 879–903 (2003). https://doi.org/10.1037/0021-9010.88.5.879
30. Shute, V.J.: Focus on formative feedback. Review of Educational Research **78**(1), 153–189 (2008). https://doi.org/10.3102/0034654307313795
31. van Sonderen, E., Sanderman, R., Coyne, J.C.: Ineffectiveness of reverse wording of questionnaire items: Let's learn from cows in the rain. PLoS ONE **8**(7), e68967 (2013). https://doi.org/10.1371/journal.pone.0068967
32. Venkatesh, V., Morris, M.G., Davis, G.B., Davis, F.D.: User acceptance of information technology: Toward a unified view. MIS Quarterly **27**(3), 425–478 (2003). https://doi.org/10.2307/30036540
33. Winstone, N.E., Nash, R.A., Parker, M., Rowntree, J.: Supporting learners' agentic engagement with feedback: A systematic review and a taxonomy of recipience processes. Educational Psychologist **52**(1), 17–37 (2017). https://doi.org/10.1080/00461520.2016.1207538
34. Zhang, Z., Dong, Z., Shi, Y., Price, T., Matsuda, N., Xu, D.: Students' perceptions and preferences of generative artificial intelligence feedback for programming. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 38, pp. 23250–23258 (2024). https://doi.org/10.1609/aaai.v38i21.30372
35. Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E.P., Zhang, H., Gonzalez, J.E., Stoica, I.: Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In: Advances in Neural Information Processing Systems. vol. 36, pp. 46595–46623 (2023)